Corrections Updates V1.md

Corrections & Updates V1

Project: ProcureBuddy (Procurement Services — FIX V1)
Date: 2026-08-01
Scope: Fix pass covering Purchase Order creation, PO document export, the supplier portal's RFQ bidding flow, quotation attachments, PO acknowledgement, PO shipment tracking, connecting GRN ↔ Invoice recording with a corrected 3-way match algorithm, Blanket PO creation, manual quotation entry, RFQ page layout, and the vendor registration approval loop.

All changes were verified after implementation with `tsc --noEmit` (TypeScript) and `eslint` on every touched file — no new type errors or lint regressions were introduced by any of the items below. Where a database schema change was involved, both the `company_default.db` template and the live `company_veltrix.db` tenant database were updated (see Database Migration Notes).

---

1. Purchase Order Creation — Line Item Description ("Scope of Supply")

Problem: When building a PO's line items, there was no way to add a free-text description to a selected item/service — only item, quantity, and unit price.

Change:

- Added a `description` field to `POItem`.
- In the **New PO** wizard (Line Items step), once an item/service is selected a **Description** textarea appears, pre-filled from that item's catalogue description and fully editable per PO line.
- The description now displays in the Step 3 draft preview and on the saved PO's detail page under the item name.
- Included in the Excel PO export (falls back to the service scope-of-work if no description was entered).

Files changed: `src/types/index.ts`, `src/components/Modals.tsx`, `src/components/PurchaseOrdersPage.tsx`, `src/utils/poExcelExport.ts`

---

2. PO PDF Export — Pagination & Full Content

Problem: The PO PDF export had no pagination logic at all — the line-items table was hand-drawn with a fixed row height, item names were truncated to a single line with no ellipsis, and the footer was hardcoded to "Page 1 of 1" regardless of how much content there actually was. A PO with many line items (or long descriptions, after item 1 above) would visually overflow and overlap the footer.

Change:

- Added `ensureSpace()` / `startNewPage()` helpers that check remaining vertical space before drawing the line-items table, totals block, remarks box, or signature blocks, and start a new page (with a lightweight "continued" header and a re-drawn table header row) whenever content would overrun the footer.
- Row height is now computed dynamically from the wrapped item name, description, and service billing/duration text — nothing is truncated anymore.
- The footer (legal text + page number) is now drawn in a final pass over every generated page, so it correctly reads **"Page X of Y"**.

Files changed: `src/utils/poPdfExport.ts`

Known limitation (not addressed): the async "PDF magic-header" validation check still runs after the download has already fired, so it remains a no-op safeguard rather than a real validation gate.

---

3. Supplier Portal — RFQ Bid Submission ("Submit Proposal") Fix

Problem: On the supplier portal's Tenders tab, the **Submit proposal** button on a live RFQ had no click handler at all — it was completely non-functional. Suppliers had no way to respond to a published tender.

Change:

- Added a proper `BidSubmissionModal` (local to the supplier portal, following the file's existing self-contained modal pattern) with per-line-item unit price and lead time inputs, payment terms, bid validity period, a technical/compliance declaration, notes, and a live running total.
- Wired the button to open this modal; submission builds a real `Quotation` record via the existing `addQuotation` mutation, so it flows into the buyer-side RFQ/Quotations pages exactly like any other bid.
- Added a UX guard: once a supplier has submitted a proposal for an RFQ, the button becomes a disabled "Proposal submitted" state instead of allowing duplicates.

Files changed: `src/components/SupplierPortalPage.tsx`

---

4. Supplier Portal — Quotation Document Attachment

Problem: Suppliers had no way to attach their own quotation document (PDF/DOC/XLS) when submitting a proposal — only the structured price/terms fields could be submitted.

Change:

- Added a real file picker to the Bid Submission modal ("Attach your quotation document") accepting PDF/DOC/XLS/image files. It captures the actual selected file's real name and size.
- Added `quotationFileName` / `quotationFileSize` to the `Quotation` record (types + Prisma schema + both DB files).
- The buyer-side Quotations page now shows a 📎 badge with the attached filename/size on each quote card, so procurement can see that a document was attached.

Files changed: `src/types/index.ts`, `prisma/schema.prisma`, `src/components/SupplierPortalPage.tsx`, `src/components/QuotationsPage.tsx`

Known limitation (by design, not a bug to fix later without a larger project): this application has **no file storage backend anywhere** — none of its "upload" features (payment receipts, compliance documents, PO documents, and now quotation attachments) persist actual file bytes to a server. This captures and displays the *real* filename/size from the browser, but the file content itself is not stored or downloadable later. Making the attachment actually retrievable by the buyer requires adding real file storage (e.g. local disk or S3) as a separate, larger piece of work.

---

5. Purchase Order Acknowledgement — Full Rework

Problem: Once a PO was issued, the supplier's "Acknowledge PO" action was a single blind click that only stamped a timestamp — no record of who acknowledged it, whether they accepted it as issued or had exceptions, or what delivery date they were actually confirming. Worse, none of this (not even the fact of acknowledgement) was visible anywhere in the internal procurement team's PO page.

Change:

- `PurchaseOrder` gained `acknowledgedBy`, `acknowledgementStatus` (`Acknowledged` / `Acknowledged with Exceptions`), `acknowledgedDeliveryDate`, and `acknowledgementNotes` (types, Prisma schema, both DB files, `ACKNOWLEDGE_PO` mutation, and `AppContext.acknowledgePO`, which now accepts an optional details object).
- **Supplier portal:** every single-PO "Acknowledge" entry point (detail drawer, table row, mobile card) now opens an `AcknowledgePOModal` capturing the acknowledger's name, an accept-as-issued vs. accept-with-exceptions decision, the confirmed delivery date (flagged if it differs from the originally requested date), and comments. The bulk "Acknowledge all" action still fast-paths multiple POs at once but now persists sensible real defaults instead of a bare timestamp.
- **Procurement team side (`PurchaseOrdersPage.tsx`)**, previously showed nothing about acknowledgement at all:
  - Added an acknowledgement badge in the PO detail header (green "Acknowledged" / amber "Acknowledged w/ Exceptions" / amber "Awaiting Acknowledgement").
  - Added a full "Supplier Acknowledgement" card on the PO detail page (who, when, confirmed date vs. requested date, comments).

    - Added an "Ack." column with an icon status indicator to the main PO list table.
- Every acknowledgement now also writes an audit log entry.

Files changed: `src/types/index.ts`, `prisma/schema.prisma`, `src/app/api/data/mutate/route.ts`, `src/context/AppContext.tsx`, `src/components/SupplierPortalPage.tsx`, `src/components/PurchaseOrdersPage.tsx`

---

6. Supplier Portal — PO PDF Download

Problem: Suppliers had no way to download the official PO document once it was issued to them.

Change: Added a **Download PDF** action to the PO detail drawer and to each PO row (table and mobile card views), reusing the same PDF generator used internally by procurement (`poPdfExport.ts`, including the pagination fix from item 2). Gated to `deliveryStatus !== 'Draft'` — i.e. only available once the PO has actually been issued.

Files changed: `src/components/SupplierPortalPage.tsx`

---

7. Supplier Portal — Shipment Update Fix

Problem: In the PO detail drawer, the "Update shipment details" form (tracking number, carrier, estimated delivery) was passing its data as an object to `updateShipment`, but the underlying function expected three separate positional string arguments. Because the surrounding code was loosely typed (`useApp() as any`), this went undetected by TypeScript — at runtime it silently corrupted the data (carrier came through as `undefined`, tracking number as a stringified object), and the "estimated delivery" date the supplier entered was discarded entirely (no database column existed for it).

Change:

- Changed `AppContext.updateShipment` to accept a single details object (`{ trackingNumber, carrier, estimatedDelivery? }`) instead of positional arguments — matching what the form was already trying to send.
- Added a `shipmentEta` column so the estimated delivery date entered by the supplier is actually persisted (types + Prisma schema + both DB files).
- Added basic validation: the Confirm button now requires both tracking number and carrier before it's enabled (previously it could be submitted empty).
- Also fixed a second, currently-unreachable call site (`Modals.tsx`'s orphaned `ShipmentConfirmationModal`) so it stays type-correct, even though nothing in the app currently opens it.
- Surfaced the captured carrier / tracking number / shipment ETA in two places it was previously invisible: the supplier portal drawer's summary tiles, and the procurement team's PO detail page (which had no shipment visibility at all before this).

- Every shipment update now also writes an audit log entry.

Files changed: `src/types/index.ts`, `prisma/schema.prisma`, `src/app/api/data/mutate/route.ts`, `src/context/AppContext.tsx`, `src/components/Modals.tsx`, `src/components/SupplierPortalPage.tsx`, `src/components/PurchaseOrdersPage.tsx`

---

8. Connecting GRN ↔ Invoice Recording

Problem: Recording an invoice (`InvoicesPage.tsx`) had no relationship to the GRN process at all — it only asked for a PO reference and a single manually-typed total amount, with `lineItems: []` hardcoded ("simplified for demo"). There was also no way to attach commercial or shipping documents (packing list, BL/AWB, MTC, COO, commercial invoice) when recording either a GRN or an invoice.

Change:

- **Record Invoice** (`InvoicesPage.tsx`'s `NewInvoiceModal`, rewritten): selecting a PO now also offers a **GRN selection**, filtered to that PO's `Approved` GRNs. Picking a GRN auto-populates the invoice's line items from the GRN's *accepted* quantities (falls back to the PO's ordered quantities if no GRN is available/selected) — every line's quantity and unit price is directly editable, with live-recalculating line and grand totals. Added an optional "Other Charges" field (tax/freight) on top of the line-item subtotal. The due date is now auto-suggested by parsing the PO's actual payment terms (e.g. "Net 45") instead of a hardcoded 30 days, and stays editable. `Invoice` gained a `grnId?` field for traceability.
- **Attachments in both flows:** built a shared `DocumentAttachmentsEditor` component, used in both `GRNPage.tsx`'s "New GRN" form and the Invoice form — add any number of documents, each with a category (Delivery Note, Packing List, BL/AWB, MTC, COO, Invoice) and a real file picker capturing the actual filename/size. Persisted through the existing document system (`AppDocument`).
- Fixed the dead "View Details" button on the invoice list — it now opens a real detail view (PO/GRN reference, 3-way match status, full line items, attached documents).
- `GRNDetail` also now shows documents attached against its PO.

Files changed: `src/types/index.ts`, `prisma/schema.prisma`, `src/components/InvoicesPage.tsx`, `src/components/GRNPage.tsx`, `src/components/DocumentAttachmentsEditor.tsx` (new, shared), `src/utils/formatFileSize.ts` (new, shared)

Known limitation: same as item 4 — no file storage backend exists anywhere in this app, so attachment file *bytes* are never persisted, only real filename/size metadata.

---

9. 3-Way Match Correction — Multiple GRNs / Multiple Invoices per PO

Problem: Discovered while building item 8. The 3-way match engine (`runThreeWayMatch` in `mutate/route.ts`) picked an *arbitrary* Approved GRN (`db.gRN.findFirst({ where: { poId, status: 'Approved' } })`) and an *arbitrary* invoice

(`db.invoice.findFirst({ where: { poId } })`) for a PO — not necessarily the ones actually involved in whatever event triggered the recalculation. Any PO with more than one GRN (partial deliveries) or more than one invoice (partial billing) could get an incorrect, effectively random match result, and other invoices on the same PO were never (re)evaluated at all.

Change:

- Each invoice now matches against **its own linked GRN** (`invoice.grnId`, from item 8), falling back to "any Approved GRN for the PO" only for older/unlinked invoices.
- `runThreeWayMatch(poId, invoiceId?)`: passing a specific `invoiceId` (used right after creating/submitting that invoice) matches only that invoice; omitting it (used after a GRN is approved, since newly-available accepted quantities can affect any invoice already recorded against the PO) re-matches **every** invoice on the PO.
- The PO's own `matchStatus` is now a real aggregate across all its invoices (worst-status-wins: Variance > Missing GRN > Pending > Full Match), instead of reflecting whichever invoice the database happened to return first.
- Threaded an optional `invoiceId` through `PERFORM_MATCH` → `AppContext.performMatch(poId, invoiceId?)` → its call site in `InvoicesPage.tsx`. Also fixed the pending-match dedup cache in `AppContext` (was keyed by `poId` alone — now `poId:invoiceId` — so checking two different invoices on the same PO no longer clobber each other's in-flight state).

Files changed: `src/app/api/data/mutate/route.ts`, `src/context/AppContext.tsx`, `src/components/InvoicesPage.tsx`

---

10. Blanket PO Creation, Manual Quotation Entry, and RFQ Widget Layout

Problem (a): "New Blanket Agreement" on `BlanketsPage.tsx` opened a completely empty modal — `'newBlanket'` had no case registered in `Modals.tsx`'s dispatcher, so there was no way to create a Blanket PO from the UI at all.

Change (a): Added `AddBlanketModal` (supplier, category scope, total ceiling, currency, valid from/to, department, project) and registered it under `newBlanket`. Uses the existing `addBlanket()` mutation, which already worked correctly server-side.

Problem (b): There was no way to manually log a quotation received outside the supplier portal (e.g. by email) — the one modal built for this (`NewQuotationModal`) was dead code, unreachable from any button, and hardcoded to a demo supplier. There was also no way to attach the email attachment itself, and the Quotations module had no search, filter, or sort — just a flat, unordered list of RFQs.

Change (b):

- Resurrected `NewQuotationModal`: added a required supplier selector (restricted to the RFQ's invited suppliers when it has any), a real file attachment field (reusing the `quotationFileName` / `quotationFileSize` fields added earlier), and removed the hardcoded demo supplier.

- Added an **"Add Quotation Manually"** button to the RFQ detail page's Bid Inbox tab (`RFQPage.tsx`, gated to the same permission as RFQ creation) that opens it.
- `QuotationsPage.tsx`'s RFQ list view now has a search box (RFQ # or title), a status filter, and sortable RFQ #/Title/Closing Date/Quotes-Received columns.

Problem (c): The RFQ page's 4 KPI bubbles (Active RFQs, Bids Received, Tenders Awarded, Draft Tenders) rendered stacked vertically instead of in a row.

Change (c): Root cause was a CSS class name bug: `RFQPage.tsx` used `className="grid grid-4"`, but `globals.css` only defined `.lg\:grid-4` (with the `lg:` literally baked into the class name) — plain `.grid-4` didn't exist, so the grid silently fell back to a single column. Added a proper `.grid-4` class matching the existing `.grid-2` / `.grid-3` pattern (including the same responsive breakpoints) and switched `RFQPage.tsx` to use it.

Files changed: `src/components/Modals.tsx`, `src/components/RFQPage.tsx`, `src/components/QuotationsPage.tsx`, `src/app/globals.css`

---

11. Vendor Registration Approval Loop (Supplier Portal → Procurement Team)

Problem: When a new supplier self-registered via the Vendor Portal, the procurement team received no usable alert and had no way to review, approve, or reject the request. A notification was technically created but mislabeled (`source: 'Document'`, no dedicated "Supplier" category), didn't link anywhere when clicked, and — critically — there was no Approve/Reject action anywhere in the app. Even worse, the registration form never collected a password, so a self-registered vendor could never actually log in, even if someone had manually flipped their status to "Active" directly in the database.

Change:

- Added a proper `'Supplier'` notification source/type. The registration notification is now correctly categorized and clicking it (from the notification bell or the Notification Center) jumps straight to that supplier's record.
- Added two new server-side actions: **Approve** (staff sets an initial login password — hashed correctly server-side with `bcrypt`, the same way seeded accounts are — which activates the supplier's status and portal access) and **Reject** (records a required reason as a note on the supplier). Both are logged to the audit trail.
- The Suppliers page now shows a **"Pending Vendor Registrations"** banner whenever any request is awaiting review, and a status badge (Pending Approval / Rejected) on both the supplier list and detail view.
- Opening a pending supplier's record now shows a full **Registration Review** panel — the financial statements, project history, and reference documents they submitted — with Approve/Reject actions directly on it. On approval, the credentials are displayed once so staff can relay them to the vendor (this app has no automated email-sending, so that hand-off is manual by design, not an oversight).

- Once approved, the supplier is immediately a normal, active entry in the supplier register — no separate "activation" step needed.

Files changed: `src/types/index.ts`, `src/app/api/data/mutate/route.ts`, `src/context/AppContext.tsx`, `src/components/VendorRegistrationForm.tsx`, `src/components/NotificationsPage.tsx`, `src/components/Sidebar.tsx`, `src/components/SuppliersPage.tsx`

Known limitation (flagged, not fixed in this pass): the supplier portal's own self-service "Change Password" feature still writes the new password as plaintext (no hashing) — inconsistent with the new Approve flow above. A supplier changing their own password today would end up locked out. Worth a follow-up fix.

Database Migration Notes

This app uses **one physical SQLite file per tenant** (`databases/company_<tenant>.db`), not a single shared database — `prisma db push` only targets the datasource configured for the CLI (`company_default.db`), so every schema change made in this pass had to be applied a second time, directly, to `company_veltrix.db` (the only tenant currently seeded and used by login) via a plain `ALTER TABLE ... ADD COLUMN` — non-destructively, preserving existing data. Anyone spinning up an additional tenant database, or deploying this to a new environment, needs to run the equivalent of:

```
npm run db:push        # applies schema to company_default.db (and any new tenant cloned from it)
npx prisma generate    # regenerates the Prisma client
```

If a tenant database already exists (was cloned *before* one of the schema changes above), it will be missing the newly added columns and needs the same `ALTER TABLE` treatment applied to `company_veltrix.db` in this session:

Column	Table
`quotationFileName`, `quotationFileSize`	`Quotation`
`acknowledgedBy`, `acknowledgementStatus`, `acknowledgedDeliveryDate`, `acknowledgementNotes`	`PurchaseOrder`
`shipmentEta`	`PurchaseOrder`
`grnId`	`Invoice`

The dev server must also be restarted after `npx prisma generate` — a running Next.js dev process keeps the previously-generated Prisma client in its module cache and won't pick up new columns until restarted.

Verification Performed

- `tsc --noEmit` after every change — zero new type errors introduced (one pre-existing, unrelated `next.config.ts` typing issue predates this work).
- `eslint` on every touched file after every change — no new errors or warnings; all remaining findings are pre-existing patterns in the codebase (extensive use of `any`, a few components created during render, etc.) that predate this fix pass.
- Local dev server (`npm run dev`) restarted and confirmed running clean after each database schema change.

Not Yet Tested End-to-End in a Live Browser Session

The following should be manually clicked through to confirm the full round trip:

1. Create a PO with a line-item description → verify it appears on the draft preview, the saved PO detail page, and the PDF/Excel export.
2. Submit a bid on a published RFQ with an attached document → confirm it shows up correctly on the buyer's Quotations page with the attachment badge.
3. Acknowledge a PO with exceptions and a different confirmed delivery date → verify the procurement team sees the exception flag, comments, and date discrepancy on the PO page.
4. Download a PO PDF from the supplier portal.
5. Update shipment details (tracking, carrier, estimated delivery) from the supplier portal → confirm the values now persist correctly and appear on both the portal and the procurement team's PO detail page.
6. Approve a GRN, then record an invoice against that PO selecting that GRN → confirm line items auto-populate from accepted quantities, attach a document, and verify the invoice reaches "Full Match" when quantities/amounts line up (and "Variance" when you deliberately edit a quantity to not match).
7. Create a second GRN and a second invoice against the same PO → confirm each invoice matches against its own linked GRN independently, and that the PO's own match status reflects the worst status across both invoices.
8. Create a new Blanket Agreement from the Blanket POs page → confirm the form works end-to-end and the new agreement appears in the list with the correct ceiling/currency/validity.
9. Open an RFQ, go to Bid Inbox → "Add Quotation Manually" → record a quotation with a supplier and an attached document → confirm it appears in the Bid Inbox and in the Quotations module, and that the 📎 badge shows on the quote card.
10. On the Quotations module's RFQ list, search by an RFQ number, filter by status, and click the sortable column headers → confirm results update correctly.
11. Open the RFQ/PR page and confirm the 4 KPI bubbles at the top now render in a single horizontal row (not stacked) at normal desktop width, and reflow to fewer columns on a narrow window.
12. Register a new vendor via the Vendor Portal → confirm a notification appears for staff, clicking it jumps to that supplier's record, the Suppliers page shows the "Pending Vendor Registrations" banner, and the

Registration Review panel shows the submitted financials/project history. Approve it with a password → confirm the vendor can then log in at `/portal` with that password. Try the Reject path too, with a reason.